AI-Powered Dynamic Pricing Engine for E-commerce

Project Overview

The project is a Python application that recommends product prices from forecast demand, inventory levels, competitor prices and sudden demand changes. Results are presented in an interactive Streamlit dashboard.

Problem Statement

Manual pricing reacts slowly to changes in demand, stock and competitor prices. This can leave popular products at risk of stockout, slow-moving products overstocked and prices uncompetitive. The project provides data-driven, explainable price recommendations to address these issues.

Objectives

- Forecast product demand from historical sales data.
- Recommend price changes together with the reasons behind them.
- Identify products at risk of stockout and products with excess stock.
- Detect unusual demand surges that resemble flash-sale patterns.

Technologies Used

Python (targeted at version 3.14.3); Streamlit (dashboard); Pandas and NumPy (data processing); Scikit-learn (Gradient Boosting Regressor and Linear Regression); Plotly (interactive charts); unittest (testing).

Key Features

- **Dynamic pricing:** combines forecast demand, days of stock cover, competitor price gap and flash-sale signals; changes are limited to ±15% (±20% in a flash sale).
- **Demand forecasting:** one Gradient Boosting model per product, evaluated on a 14-day hold-out period; actual and predicted demand are charted.
- **Revenue and inventory analysis:** classifies products as Stockout Risk, Excess Stock or Healthy.
- **Flash-sale detection:** compares the last 3 days with the previous 28 days using a z-score and demand ratio.
- **Data handling:** sample (synthetic) dataset, CSV upload and column/value validation.

System Workflow

Data input (sample or uploaded CSV) → validation and cleaning → demand forecasting → flash-sale detection → price recommendation → revenue and inventory analysis → dashboard display and CSV download of recommendations.

Implementation and Results

The dashboard has five tabs: Dynamic Pricing, Demand Forecast, Revenue & Inventory, Flash Sale Detection, and Data & Method. For each product it shows the current price, recommended price, percentage change, action (Increase, Decrease or Hold) and written reasons. On the synthetic sample dataset (8 products, 120 days), the system raised prices for low-stock and flash-sale products, lowered prices for an overstocked product and a product priced above its competitor, and flagged one flash-sale pattern. Ten automated tests covering the four core features passed.

Conclusion

The project delivers an end-to-end, explainable pricing workflow that combines machine learning forecasting with business rules. Learning outcomes include feature engineering for time series, hold-out model evaluation, data validation, dashboard development and automated testing. The system was tested only on synthetic data and uses no cost or margin data; its real-world revenue impact has not been measured.